LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 10 – PENCARIAN NILAI MAX/MIN

NAMA : ANDHIKA NUR PRATAMA

NIM : 109082500056

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pencarian nilai ekstrim adalah proses menemukan nilai terbesar (maksimum) atau terkecil (minimum) dari sekumpulan data dalam array, yang bekerja secara sekuensial dengan kompleksitas waktu O(n). Algoritmanya dimulai dengan menjadikan elemen pertama sebagai nilai ekstrim sementara, lalu memeriksa setiap elemen berikutnya satu per satu, jika ditemukan elemen yang lebih kecil atau lebih besar, nilai ekstrim diperbarui, hingga semua data selesai diperiksa. Fungsi pencarian bisa mengembalikan nilai langsung menggunakan return, atau mengembalikan indeks posisi nilai ekstrim agar data lain yang berkaitan tetap bisa diakses. Pendekatan ini juga berlaku pada tipe data terstruktur (struct), misalnya mencari mahasiswa dengan IPK tertinggi atau balita dengan berat terkecil, dengan cara membandingkan salah satu field dari struct tersebut. Apabila dibutuhkan lebih dari satu nilai hasil (seperti min dan maks sekaligus), digunakan parameter pointer agar nilai asli di fungsi pemanggil dapat diubah langsung dari dalam fungsi.

B. Guided

1. [Guided 1]

```
package main

import "fmt"

type arrInt [2023]int

func terkecil_1(tabInt arrInt, n int) int {
    var min int = tabInt[0]
    var j int = 1

    for j < n {
        if min > tabInt[j] {
            min = tabInt[j]
        }
        j = j + 1
    }
    return min
}

func terkecil_2(tabInt arrInt, n int) int {
    var idx int = 0
    var j int = 1

    for j < n {
        if tabInt[idx] > tabInt[j] { // [9, 8, 5, 6]
            idx = j
```

```
        }
        j = j + 1
    }
    return idx
}

func main() {
    var n int
    var arr arrInt

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan elemen array :")

    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
    }

    min := terkecil_1(arr, n)
    idx := terkecil_2(arr, n)

    fmt.Println("Nilai terkecil: ", min)
    fmt.Println("Indeks nilai terkecil: ", idx)
}
```

Output :

```
PS D:\109082500056_Andhika-Nur-Pratama> go run "d
ma\Pertemuan10_Modul 10\Guided\Guided 1\guided1.g
Masukkan jumlah data: 3
Masukkan elemen array :
2
4
5
Nilai terkecil:  2
Indeks nilai terkecil:  0
PS D:\109082500056_Andhika-Nur-Pratama>
```

2. [Guided 2]

```
package main
```

```go

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

type arrMahasiswa [3]mahasiswa

//data index 0 = nama, nim, IPK
//data index 1 = nama, nim, ipk
//data index 2 = nama, nim, ipk

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0] //nilai awal
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0] //nilai awal
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

```

```go
func main() {
    var arrMhs arrMahasiswa
    var i int

    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("INPUT DATA MAHASISWA INDEX KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkkan nim : ")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("================================")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])

    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("DATA IPK TERKECIL DAN TERBESAR")
    fmt.Println("=== IPK TERBESAR ===")
    fmt.Println("IPK Terbesar = ", mhsMaks.IPK)
    fmt.Println("Atas nama = ", mhsMaks.nama)
    fmt.Println("nim = ", mhsMaks.nim)
    fmt.Println("ditemukan pada indeks ke-", idxMaks)
    fmt.Println()
    fmt.Println("=== IPK TERKECIL ===")
    fmt.Println("IPK Terkecil = ", mhsMin.IPK)
    fmt.Println("Atas nama = ", mhsMin.nama)
    fmt.Println("nim = ", mhsMin.nim)
    fmt.Println("ditemukan pada indeks ke-", idxMin)
}
```

Output :

```
PS D:\109082500056_Andhika-Nur-Pratama> go
ma\Pertemuan10_Modul 10\Guided\Guided 2\Min
INPUT DATA MAHASISWA INDEX KE- 0
Masukkan nama : Nomi
Masukkkan nim : 8745676
Masukkan IPK : 4.0
================================
INPUT DATA MAHASISWA INDEX KE- 1
Masukkan nama : Vivian
Masukkkan nim : 5674657
Masukkan IPK : 4.1
================================
INPUT DATA MAHASISWA INDEX KE- 2
Masukkan nama : Sayami
Masukkkan nim : 6894722
Masukkan IPK : 4.5
================================

DATA IPK TERKECIL DAN TERBESAR
=== IPK TERBESAR ===
IPK Terbesar =  4.5
Atas nama =  Sayami
nim =  6894722
ditemukan pada indeks ke- 2

=== IPK TERKECIL ===
IPK Terkecil =  4
Atas nama =  Nomi
nim =  8745676
ditemukan pada indeks ke- 0
PS D:\109082500056_Andhika-Nur-Pratama>
```

C. Unguided

1. Soal Unguided 1
   [DataBeratKelinci]

   Jawaban :

```
package main

import "fmt"

const NMAX = 1000

type arrKelinci [NMAX]float64

func MinMax(data arrKelinci, n int, max, min *float64) {
    *min = data[0]
    *max = data[0]

    for i := 1; i < n; i++ {
        if data[i] < *min {
            *min = data[i]
```

```go
        }
        if data[i] > *max {
            *max = data[i]
        }
    }
}

func main() {
    var n int
    var data arrKelinci
    var max, min float64

    fmt.Print("Banyak anak kelinci: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Berat anak kelinci ke-%d: ", i+1)
        fmt.Scan(&data[i])
    }

    MinMax(data, n, &max, &min)
    fmt.Printf("Berat terkecil: %.2f\n", min)
    fmt.Printf("Berat terbesar: %.2f\n", max)
}
```

Output :

```
PS D:\109082500056_Andhika-Nur-Pratama> go ru
ma\Pertemuan10_Modul 10\Unguided\Unguided-1\D
Banyak anak kelinci: 2
Berat anak kelinci ke-1: 5.3
Berat anak kelinci ke-2: 7.8
Berat terkecil: 5.30
Berat terbesar: 7.80
PS D:\109082500056_Andhika-Nur-Pratama>
```

Penjelasan :

[Program ini bertujuan mencari berat terkecil dan terbesar dari sejumlah anak kelinci menggunakan array bertipe float64 berkapasitas 1000 elemen. Di fungsi main, pengguna diminta memasukkan jumlah kelinci lalu berat masing-masing kelinci yang disimpan ke dalam array satu per satu. Setelah semua data diinput, fungsi MinMax dipanggil dengan mengirimkan alamat variabel min dan max menggunakan & agar nilainya bisa diubah langsung dari dalam fungsi. Di dalam MinMax, elemen pertama dijadikan nilai ekstrim awal, lalu setiap elemen berikutnya dibandingkan satu per satu, jika ada yang lebih kecil maka *min diperbarui, jika ada yang lebih besar maka *max diperbarui. Setelah

semua elemen selesai diperiksa, hasil berat terkecil dan terbesar ditampilkan dengan format 2 angka di belakang koma menggunakan %.2f]

2. Soal Unguided 2
   [tariflkan]

   Jawaban :

```
package main

import "fmt"

const NMAX = 1000

type arrIkan [NMAX]float64

func hitungWadah(tab arrIkan, x, y int) {
    jumlahWadah := x / y
    if x%y != 0 {
        jumlahWadah++
    }

    totalPerWadah := make([]float64, jumlahWadah)

    for i := 0; i < x; i++ {
        wadah := i / y
        totalPerWadah[wadah] += tab[i]
    }

    for i := 0; i < jumlahWadah; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Printf("%.2f", totalPerWadah[i])
    }
    fmt.Println()

    totalSemua := 0.0
    for i := 0; i < jumlahWadah; i++ {
        totalSemua += totalPerWadah[i]
    }
    rataRata := totalSemua / float64(jumlahWadah)
    fmt.Printf("%.2f\n", rataRata)
}

func main() {
    var arr arrIkan
    var x, y int

    fmt.Print("Banyak ikan yang dijual: ")
```

```
    fmt.Scan(&x)
    fmt.Print("Banyak ikan per wadah: ")
    fmt.Scan(&y)

    for i := 0; i < x; i++ {
        fmt.Printf("Berat ikan ke-%d: ", i+1)
        fmt.Scan(&arr[i])
    }

    hitungWadah(arr, x, y)
}
```

Output :

```
PS D:\109082500056_Andhika-Nur-Pratama> go r
ma\Pertemuan10_Modul 10\Unguided\Unguided-2
Banyak ikan yang dijual: 3
Banyak ikan per wadah: 5
Berat ikan ke-1: 6.8
Berat ikan ke-2: 5.9
Berat ikan ke-3: 7.7
20.40
20.40
PS D:\109082500056_Andhika-Nur-Pratama>
```

Penjelasan :

[Program ini bertujuan menghitung total berat ikan per wadah dan rata-rata berat antar wadah, di mana pengguna memasukkan jumlah ikan x dan kapasitas per wadah y, lalu berat masing-masing ikan yang disimpan ke dalam array arrIkan bertipe float64. Di dalam fungsi hitungWadah, jumlah wadah dihitung dengan x/y dan jika ada sisa ikan yang tidak memenuhi satu wadah penuh maka jumlah wadah ditambah 1 menggunakan pengecekan x%y != 0. Setiap ikan kemudian dikelompokkan ke wadahnya masing-masing menggunakan rumus wadah := i/y, lalu beratnya dijumlahkan ke totalPerWadah[wadah]. Setelah semua ikan dikelompokkan, total berat tiap wadah dicetak berurutan dipisah spasi, kemudian semua total dijumlahkan dan dibagi jumlah wadah untuk mendapatkan rata-rata, yang keduanya ditampilkan dengan format 2 angka di belakang koma menggunakan %.2f.]

3. Soal Unguided 3
   [posyandu]

Jawaban :

```
package main

import "fmt"

const NMAX = 100

type arrBalita [NMAX]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]
    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    total := 0.0
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var arr arrBalita
    var n int

    fmt.Print("Masukan banyak data berat balita : ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&arr[i])
    }

    var bMin, bMax float64
    hitungMinMax(arr, n, &bMin, &bMax)
    avg := rerata(arr, n)

    fmt.Printf("Berat balita minimum: %.2f kg\n", bMin)
    fmt.Printf("Berat balita maksimum: %.2f kg\n", bMax)
```

```
    fmt.Printf("Rerata berat balita: %.2f kg\n", avg)
}
```

Output :

```
PS D:\109082500056_Andhika-Nur-Pratama> go ru
ma\Pertemuan10_Modul 10\Unguided\Unguided-3\p
Masukan banyak data berat balita : 3
Masukan berat balita ke-1: 3.3
Masukan berat balita ke-2: 4.3
Masukan berat balita ke-3: 4.5
Berat balita minimum: 3.30 kg
Berat balita maksimum: 4.50 kg
Rerata berat balita: 4.03 kg
PS D:\109082500056_Andhika-Nur-Pratama>
```

Penjelasan :

[Program ini bertujuan mencari berat terkecil, terbesar, dan rata-rata berat balita dari data yang dimasukkan pengguna, menggunakan array arrBalita bertipe float64 berkapasitas 100 elemen. Di fungsi main, pengguna diminta memasukkan jumlah balita lalu berat masing-masing balita yang disimpan ke dalam array satu per satu dengan prompt interaktif. Setelah semua data diinput, fungsi hitungMinMax dipanggil dengan mengirimkan alamat variabel bMin dan bMax menggunakan & agar nilainya bisa diubah langsung dari dalam fungsi - di dalamnya elemen pertama dijadikan nilai ekstrim awal, lalu setiap elemen berikutnya dibandingkan satu per satu sehingga *bMin diperbarui jika ada yang lebih kecil dan *bMax diperbarui jika ada yang lebih besar. Selain itu, fungsi rerata dipanggil secara terpisah untuk menjumlahkan seluruh elemen array lalu membaginya dengan jumlah balita n menggunakan float64(n) agar hasil pembagian berupa desimal. Ketiga hasil yaitu berat minimum, maksimum, dan rata-rata kemudian ditampilkan dengan format 2 angka di belakang koma menggunakan %.2f.]

D. Kesimpulan

[Ketiga program latihan pada Modul 10 sama-sama menerapkan konsep pencarian nilai ekstrim pada array, namun dengan konteks dan kompleksitas yang berbeda. Program latihan 1 merupakan penerapan paling dasar yaitu mencari berat terkecil dan terbesar dari data kelinci menggunakan dua pointer sekaligus dalam satu fungsi. Program latihan 2 lebih kompleks karena tidak hanya mencari nilai ekstrim tetapi juga mengelompokkan data ikan ke dalam wadah-wadah berdasarkan kapasitas, lalu menghitung rata-rata antar wadah. Program latihan 3 menggabungkan kedua konsep sebelumnya sekaligus, yaitu mencari nilai minimum, maksimum, dan rata-rata berat balita dengan memisahkan logika ke

dalam dua fungsi berbeda yaitu hitungMinMax dan rerata sesuai spesifikasi subprogram yang diberikan. Secara keseluruhan, ketiga program ini menunjukkan bahwa algoritma pencarian nilai ekstrim dapat diterapkan secara fleksibel — mulai dari pencarian sederhana, pengelompokan data, hingga penggunaan beberapa fungsi terpisah — dengan tetap menggunakan konsep dasar yang sama yaitu iterasi sekuensial dan pembaruan nilai ekstrim saat ditemukan data yang lebih kecil atau lebih besar.]